

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Module - I

Practical – V

Roll No.:T007	Name:Subhashree Jena
Class:TYIT	Batch:1
Date of Assignment: 21/07/26	Date/Time of Submission: 28/07/26

5. Create the following application:

a. Create a web application to demonstrate the use of different types of Cookies.

STEPS:-

- 1.Start the Application
- 2.Page Load (WebForm1.aspx.cs)
- 3.ViewState Button Click (Button1)
- 4.Create Cookie (Button2 Click) 5.Read Cookie (WebForm2.aspx.cs)
- 6.Stop the program.

CODE:-

WebForm1.aspx.cs:

```
using System; using
System.Collections.Generic; using
System.Linq; using System.Web;
using System.Web.UI; using
System.Web.UI.WebControls;
namespace Prac5c
{
    public partial class WebForm1 :
    System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if
(IsPostBack)
            {
                int ViewStateVal = Convert.ToInt32(ViewState["count"]) + 1;
                Label2.Text = "View State = "+ViewStateVal.ToString();
            }
            else
            {
                ViewState["count"] = 1;
            }
        }
        protected void Button1_Click(object sender, EventArgs e)
        {
            Label1.Text=ViewState["count"].ToString();
        }
        protected void Button2_Click(object sender, EventArgs e)
        {
            HttpCookie h=new HttpCookie("name");
            h.Value=TextBox1.Text;
            Response.Cookies.Add(h);
            Response.Redirect("WebForm2.aspx");
        }
    }
}
```

```

    }
}

```

WebForm2.aspx.cs:

```

using System; using
System.Collections.Generic; using
System.Linq; using System.Web;
using System.Web.UI; using
System.Web.UI.WebControls;
namespace Prac5c
{
    public partial class WebForm2 :
    System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if(Request.Cookies["name"] != null)
            {
                Response.Write("Welcome " + Request.Cookies["name"].Value);
            }
        }
        else
        {
            Label1.Text = "No cookie found.";
        }
    }
}

```

OUTPUT:-

Label	1	
Label	View State = 2	

View State	View State	
	Subhashree Jena	Welcome Subhashree Jena
Cookies	Cookies	Label

b. Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core. STEPS:-

- 1.Start
- 2.Configure services and database
- 3.Build application
- 4.Configure middleware and routing
- 5.Run application
- 6.Handle request in controller
- 7.Create product object
- 8.Add product to DbContext
- 9.Save changes to database
- 10.Display success message
11. Stop the program.

CODE:-

Product.cs:

```

public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Price { get; set; }
}

```

AppDbContext.cs:

```

using Microsoft.EntityFrameworkCore; public
class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext>
options)
        : base(options)
    {
    }
    public DbSet<Product> Products { get; set; }
}

```

Appsettings.json:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection":
    "Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}
```

HomeController.cs:

```
using Microsoft.AspNetCore.Mvc; public
class HomeController : Controller
{
    private readonly AppDbContext _context;
    public HomeController(AppDbContext context)
    {
        _context = context;
    }
    public IActionResult Index()
    {
        if (!_context.Products.Any())
        {
            Product p = new Product
            {
                Name = "Laptop",
                Price = 50000
            };
            _context.Products.Add(p);
            _context.SaveChanges();
        }
        return Content("Heyyy subhashree T007 Product Inserted Successfully");
    }
}
```

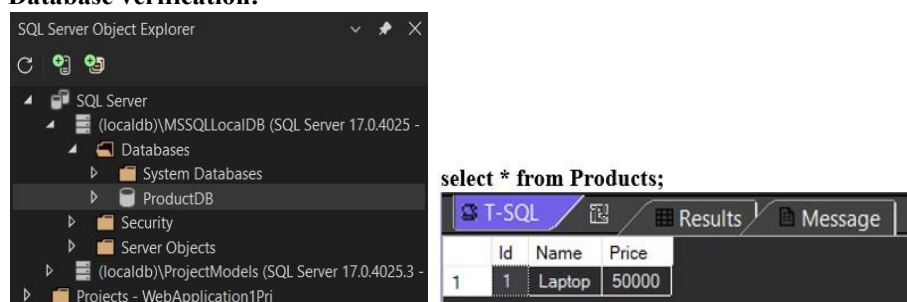
Program.cs:

```
using Microsoft.EntityFrameworkCore;
var builder = WebApplication.CreateBuilder(args);
// Add services builder.Services.AddControllersWithViews();
// Add DB Context
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
var app = builder.Build(); app.UseStaticFiles(); app.UseRouting(); // Default Route
app.MapControllerRoute( name: "default", pattern:
"{controller=Home}/{action=Index}/{id?}"); app.Run();
```

OUTPUT:-

Heyyy Subhashree T007 Product Inserted Successfully

Database verification:



The screenshot shows the SQL Server Object Explorer on the left, displaying the hierarchy: SQL Server > (localdb)\\MSSQLLocalDB (SQL Server 17.0.4025 -) > Databases > ProductDB. On the right, the T-SQL query editor shows the query 'select * from Products;' and the Results tab displays a single row with the following data:

	Id	Name	Price
1	1	Laptop	50000

c. Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on Employee table using EF Core.

STEPS:-

1. Start
2. Create Model Class
3. Configure DbContext
4. Configure Database Connection
5. Apply Migration
6. Perform CRUD Operations
7. Stop

CODE:-

EmployeesController.cs

```
using EmployeeCrud.Data;
using EmployeeCrud.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

1 reference
public class EmployeesController : Controller
{
    private readonly AppDbContext _context;

    0 references
    public EmployeesController(AppDbContext context)
    {
        _context = context;
    }

    // GET: EMPLOYEES
    3 references
    public async Task<IActionResult> Index()
    {
        return View(await _context.Employees.ToListAsync());
    }
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using EmployeeCrud.Data;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>(options =>options.UseSqlServer(builder.Configuration.GetConnectionString("AppDbContext")));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for production scenarios, see https://aka.ms/aspnetcore-hsts
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();
```

Appsettings.json

```
{
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True",
    "AppDbContext": "Server=(localdb)\\mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleActiveResultSets=true"
  }
}
```

AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;

namespace EmployeeCrud.Data
{
    7 references
    public class AppDbContext : DbContext
    {
        0 references
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
        7 references
        public DbSet<EmployeeCrud.Models.Employee> Employees { get; set; }
    }
}
```

EmployeeCrud

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <NuGetAuditSuppress Include="https://github.com/advisories/GHSA-g4vj-cjjj-v7hg" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="10.0.10">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="10.0.10" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="10.0.10">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.VisualStudio.Web.CodeGeneration.Design" Version="10.0.2" />
  </ItemGroup>

</Project>
```

Employee.cs

```
namespace EmployeeCrud.Models
{
    13 references
    public class Employee
    {
        11 references
        public int Id { get; set; }
        12 references
        public required string Name { get; set; }
        12 references
        public required string Department { get; set; }
        12 references
        public double Salary { get; set; }
    }
}
```

OUTPUT:-

EmployeeCrud Home Privacy

Create Employee

Name

Subhashree Jena

Department

BSCIT

Salary

80000

Create

[Back to List](#)

Index

[Create New](#)

Name	Department	Salary	
Subhashree Jena	BSCIT	80000	Edit Details Delete

d. Authentication and Authorization in ASP.NET Core.

STEPS:-

- 1.Create an ASP.NET Core MVC project.
- 2.Enable Cookie Authentication in Program.cs.
- 3.Create Login action in HomeController.
- 4.Verify username and password.
- 5.Redirect to Secure page after successful login.
- 6.Use [Authorize] to protect the Secure page.
- 7.Logout and redirect to Login.

CODE:-

Index.cshtml:

```
<h2>Login</h2> <form
method="post">
Username :
    <input type="text" name="username" /><br /><br />
Password :
    <input type="password" name="password" /><br /><br />
    <input type="submit" value="Login" />
</form>
<p style="color:red">
    @ViewBag.Message
</p>
```

Program.cs:

```
using Microsoft.AspNetCore.Authentication.Cookies; var builder =
WebApplication.CreateBuilder(args); builder.Services.AddControllersWithViews();
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
.AddCookie(options =>
{
    options.LoginPath =
"/Home/Index";
}); builder.Services.AddAuthorization(); var app =
builder.Build(); app.UseStaticFiles(); app.UseRouting();
app.UseAuthentication(); app.UseAuthorization();
app.MapControllerRoute( name: "default", pattern:
"{controller=Home}/{action=Index}/{id?}");
app.Run();
```

HomeController.cs:

```
using Microsoft.AspNetCore.Mvc; using
Microsoft.AspNetCore.Authorization; using
Microsoft.AspNetCore.Authentication; using
Microsoft.AspNetCore.Authentication.Cookies; using
System.Security.Claims; namespace
AuthDemo.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index(){
return View();
        }
        [HttpPost]    public async Task<IActionResult> Index(string username,
string password)
        {
            if (username == "admin" && password == "123")
            {
                var claims = new List<Claim>
                {
                    new Claim(ClaimTypes.Name, username)
                };
                var identity = new ClaimsIdentity(claims,
                    CookieAuthenticationDefaults.AuthenticationScheme);

                await HttpContext.SignInAsync(
```

```
        CookieAuthenticationDefaults.AuthenticationScheme,
new ClaimsPrincipal(identity));

        return RedirectToAction("Secure");
    }
    ViewBag.Message = "Invalid Username or Password";
return View();
}
[Authorize]    public
ActionResult Secure()
{
    return Content("Welcome! Login Successful.");
}
public async Task<ActionResult> Logout()
{
    await HttpContext.SignOutAsync(
        CookieAuthenticationDefaults.AuthenticationScheme);
return RedirectToAction("Index");
}}}
```

OUTPUT:-

Login

Username :

Password :

Welcome! Login Successful.